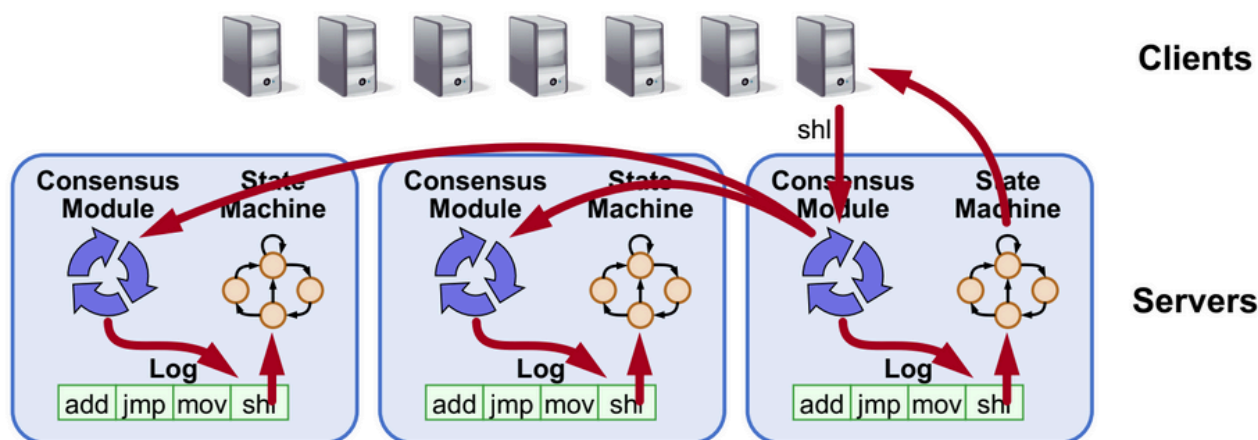


Distributed Systems - 26_11

Raft is a system that replicates a replicated state machine in eventually-synchronous systems, it is a primary backup approach.

The main problem it solves is the keeping of a **replicated log** (append only list). The clients will communicate with the primary replica, known as leader, who will run a consensus model on the system



Imagine a row of servers. On top of them, we have clients. The clients send request to the servers. Specifically, they talk to one server called the Leader.

Inside each server, there are three main components working in a loop:

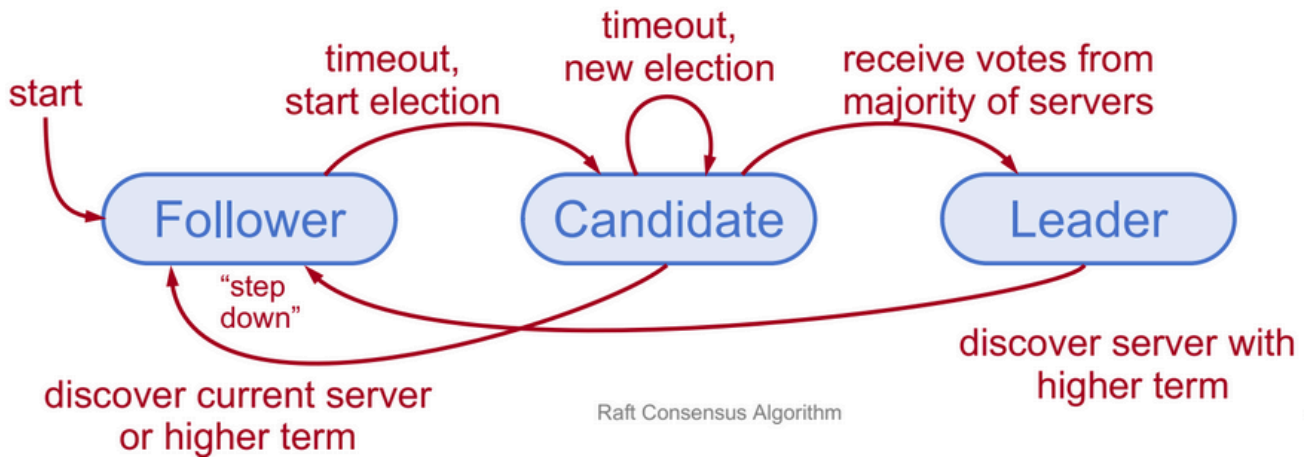
1. The consensus module, this receives the commands.
2. The log, the consensus module appends the commands here
3. The state machine, once the command is safe, the state machine executes it.

The rule is simple: Replicated log implies a replicated state machine. All servers execute the same commands in the same order. The system continues to make progress, as long as any majority of servers are up.

The normal operation happens when the leader is correct. There is a problem though, when the leader dies we need to elect a new one but we need to keep consistency on our machines. Since we are in ES and we don't have P , another module is needed to neutralise the leaders, as in ES we may think the leader is dead, but it's still alive. We need a way to tell the old leader to surrender and give its powers to a new leader. Due to this, we also need the "candidate" module, as some clients may differ in who they think is dead.

Servers specifically behave in this way, with each server having only one of these roles at a given time:

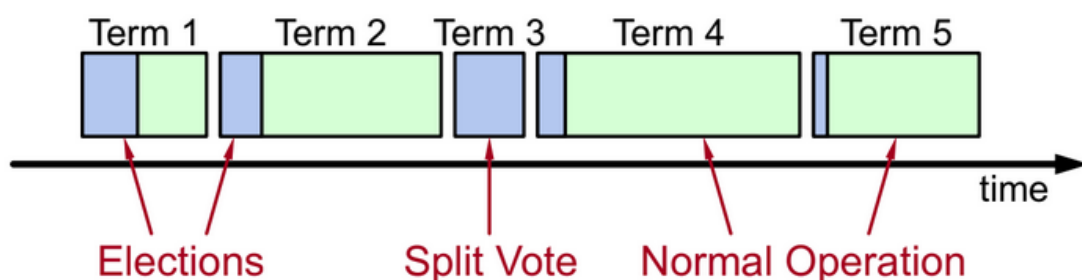
- The leader: handles all client interactions, managing log replication. There is at most one viable leader at a time.
- The follower: they are passive, they do not issue requests, they respond to incoming messages. Think of them as backups.
- The candidate: this state is temporary, used solely to elect a new leader.



Let's say we start as a follower. If a timeout occurs, it means we haven't heard anything from a leader, and in that case we start an election. We become a candidate, asking every process to vote for us. If we receive votes from a majority, we become a leader, in fact by receiving a majority we have no other candidates that can actually become a leader. This implies that the number of failures must be $< \frac{n}{2}$. However, if we discover a server with an higher term (it means a new leader has been elected), we step down, and we become a follower again. This may happen when we are correct but our channel is slow. A candidate will stop being a candidate if someone won the election.

This brings us to the concept of **terms**. Time in Raft is divided into numbered terms, with each term being associated with an integer.

Visually, imagine a horizontal time line. We have blue blocks and green blocks. The blue blocks represent elections. The green blocks represent Normal Operation, under a single leader. Crucially, terms act as a logical clock, to identify obsolete information.



We have at most one leader per term, with some terms having no leader (due to a failed election), and thus no normal operation (as we can see in term 3). This may

happen if we don't reach a majority. If this happen, we have to do a new leader election, for different terms we may have different leaders.

In the replicas memory we also have a term counter, and when we receive a term with an higher counter, we switch to it.

Let's now take a look at a summary for Raft:

- Followers: followers respond to RPCs (Remote Procedure Call, it involves sending a request and waiting for a response, such that when a server receives an RPC with a certain input, it must execute the command and respond with only the answer) from candidates and leaders. They convert into candidates if election timeout elapses without having received valid AppendEntries RPC, or granting vote to candidate.
- Candidates: They:
 - Increment currentTerm, voting for themselves.
 - Reset election timeouts.
 - Send RequestVote RPCs to all the other servers, waiting either for a majority of servers (becoming leader in this case); appendentries RPC from a new leader, thus stepping down; election timeout, thus we don't have an election resolution, and we increase the term, starting a new election; discover a higher term, stepping down.
- Leaders. They initialize nextIndex for each last log index of each follower. They send heartbeat requests to each follower, thus avoiding leader elections during idle time. They accept commands from clients, appending new entries to local log. Whenever their last log index is greater than the one of a follower, send AppendEntries RPC with log entries starting at nextIndex, updating it if succesful. If AppendEntries fails, maybe due to log inconsistency, decrement nextIndex and retry. It marks log entries as committed if stored on a majority of servers, and also at least one entry from current term is stored on a majority of servers. Step down if currentTerm changes.
- Persistent State: Each server persists the following to stable storage synchronously before responding to RPCs.
 - currentTerm: latest term server has seen (first set to 0)
 - votedFor: candidateId that received vote in current term (null if none)
 - log[], log entries.
- Log entry: a log entry consists of:
 - term: term when entry was received by leader
 - index: position of entry in the log

- command: command for state machine
- RequestVote RPC: It is invoked by candidates to gather votes. The arguments of this RPC are given by: candidateId, the id of the candidate requesting vote; term, current term of the candidate; lastLogIndex, index of candidate last log entry; lastLogTerm, term of candidate's last log entry. The results contains the term to which the candidate has to update itself, and voteGranted, which is a boolean stating if the candidate has received vote. The implementation: if $term > currentTerm$, we update our term. If term is equal to current term, votedFor is null or candidateId, and candidate's log is at least as complete as local log, we grant vote and reset election timeout.

We also have AppendEntries RPC. It's an RPC sent by the leader to followers, telling them to update their log with a new entry x . When we are a follower, we expect to receive this kind of RPC, which will only arrive obviously if a client made a request. Otherwise, the leader will send an AppendEntries RPC anyways, this time with value null, counting as an heartbeat.

When a follower sees a request, it executes a request and starts a timer. When it times-out, the follower increases the term becoming a candidate. It votes for himself and sends a requestRPC to all other servers. After this, we can have three possible scenarios:

1. We receive votes from a majority of servers, thus becoming leaders, and sending AppendEntries heartbeats to all the other servers.
2. We receive RPC from valid leader, return to follower state
3. No-one wins election (an election timeout occurs), thus we increment term, and start a new election.

Everyone will vote for the first candidate that contacts them.

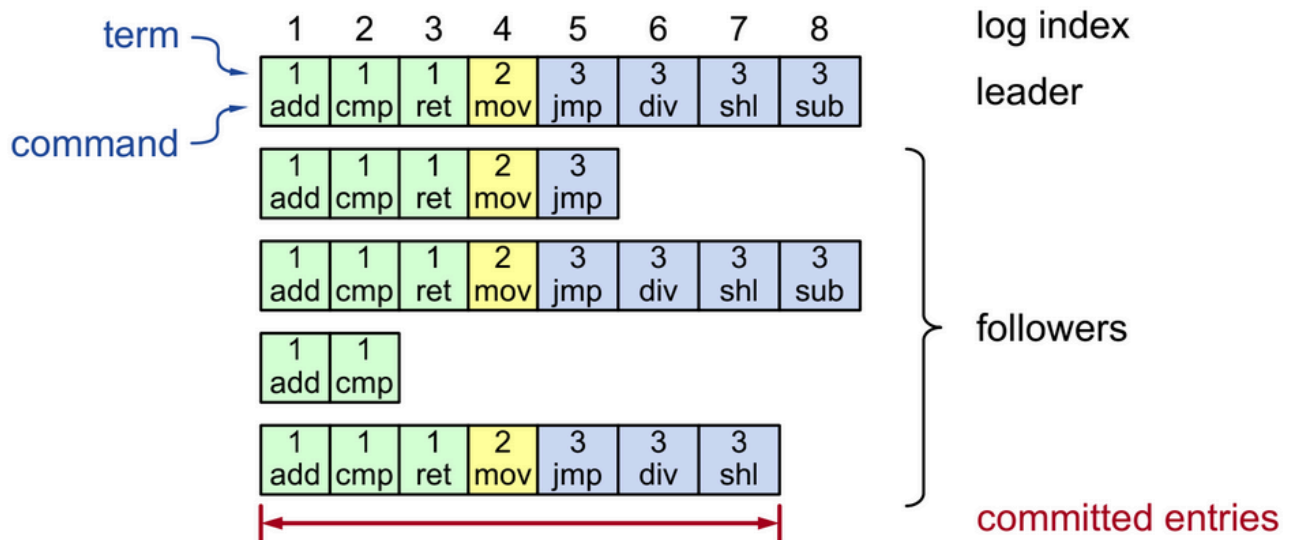
The third scenario is known as split-election (suppose we have four processes, with p_1 dying, and p_2 and p_4 trying to elect themselves. We have p_2 getting p_2 , and p_3, p_4 getting p_4). Just starting an election will put us in a locked state, that's why the timeout value is random for each candidate, so that at least one of them will win the race, getting all necessary votes.

Here are the properties:

- Safety: allow at most one winner per term. Each server will give only one vote per term, but there is the possibility of split votes, with no one winning.

- Liveness: some candidates must eventually win. To do this, we choose a timeout randomly in $[T, 2T]$. This works if $T \gg$ broadcast time, and only because we are in eventually synchronous system, so we are assuming $\exists T^*$ such that the system become synchronous forever. If we are in asynchrony, in fact, we might get split vote.

Now, what is the **log**? It's an append only list with 2 elements, the term and the command.



When the leader gets a command, it first appends to its own log, but it does not reply yet with the result. First, he has to be sure that the command is accepted by the machine, which means that even if he dies the machine will still have it. So, he sends the command to all the replicas and wait for ACKs. We don't have, however, P ! Therefore he will broadcast the command, waiting for a majority of ACKs, only then sending a response to the client. This strategy may leave, however, some processes behind (as they might be slow). In that case the leader will broadcast the command and only some of the followers will get it, but we don't care about discrepancy.

If the leader stays alive, we have a message delay of $2N$ (broadcast and ACKs). If it dies, we must account for elections. Also, when the system is asynchronous, we are unbounded.

When we get slow followers, we just continue to send the RPC call even after receiving commands (we'll have a set containing who has what).

When the leader changes, we need an agreement on the log. In Raft there is a property where if for a certain index (e.g. 5) we find two replicas for a certain term (e.g. 3), this means that the command is the same, but if for a certain replica we find a command from two terms (e.g. 3 and 4), this must mean that the entry that was on previous term was not committed, therefore we need a way to modify the

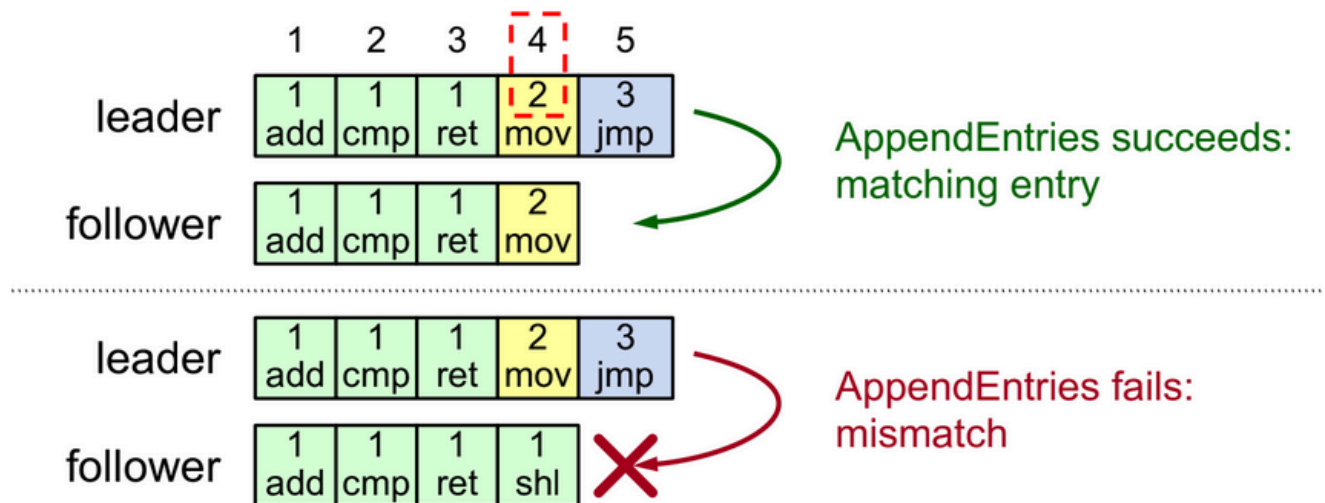
log, and keep consistency.

1	2	3	4	5	6
1 add	1 cmp	1 ret	2 mov	3 jmp	3 div
1 add	1 cmp	1 ret	2 mov	3 jmp	4 sub

Each AppendEntries RPC contains index, term of entry preceding new one.

Follower must contain matching entry, otherwise it rejects request.

It implements an induction step, ensuring coherency.



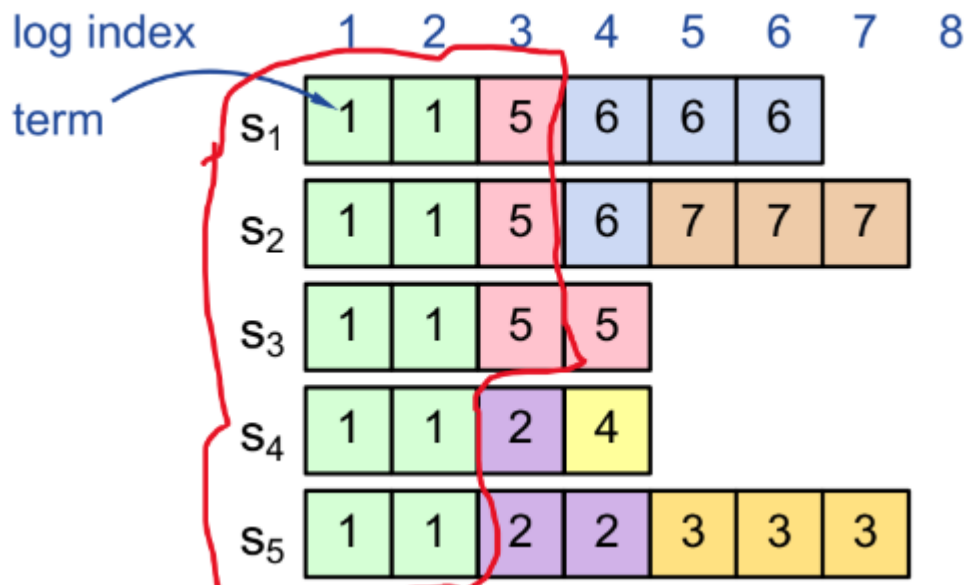
A follower will only accept a new entry if it can prove it has the exact same history as the leader up to that point. When a leader wants to append a command, it will not just send data, he will also ask the follower to check if the previous entry (4 in pic) has the same term number. If yes, he will append, and this is known as inductive step as if index 4 matches, then also previous ones have to match. In the second scenario shown above, the term n mismatches, thus leading the follower to reject the request.

When a mismatch occurs, it means that the follower has garbage data (uncommitted entries from an old failed leader), or is missing data. In order to fix this, the server has to backtrack, finding an index on which they agree, and starting overwriting its own truth from there. This is risky, as we could be overwriting a transaction that has already happened (and we can't do that). The solution is given by Raft's safety rule, which ensures that a Leader is never elected unless it already contains all committed entries. Therefore, the only entries getting overwritten are uncommitted ones that never reached a majority and were never confirmed to clients. Safety is preserved.

Now, when we elect a leader we must make sure that the followers follow the leader's entries, but we must make sure as well that we don't elect a leader that

overwrites committed changes.

We might have a situation like this with the different replicas' logs.



We should remember, at first, that the new elected leader's log is considered the truth, and it will be forced to all followers such that they'll duplicate the exact same log as the leader.

Before a leader can overwrite follower's logs, we must guarantee Safety. We cannot lose data that was already committed. If a leader decides that an entry has been already committed, then that entry has to be in every log of all future leaders, otherwise they will impose their logs which don't contain an entry that has already been committed (which would mean that there is a roll back of the state machine).

In order to achieve this, Raft imposes strict restrictions on Election and Commitment.

In fact, we cannot just choose any server as a leader, if a server missing committed entries were elected, it would overwrite other servers' log, breaking safety property. So the main constraint is that a candidate is not eligible unless its log contains all committed entries. When a candidate sends his proposal, he will send lastLogIndex and lastLogTerm as well, such that the voter will compare those values with the one he has. The voter denies the vote if its log is more complete or newer than the candidate. First we compare the term value (the higher the better), then the length of the log.

Now, we have to deal with committing entries from earlier terms. It's a specific case in which a leader has to replicate an entry that was created by a previous leader. Imagine we are trying to replicate an entry from term 2 (being now in term 4). Even if a leader manages to replicate this entry to a majority of servers, it is not safe to call it committed yet. In fact, if the leader crashes after replicating the

entry, a different server (like S_5) could still be elected leader for term 5. Because the replicated entry has low term (2), S_5 might still outrank the others during election logic and overwrite that entry.

What is the solution? A leader cannot conclude an entry from an earlier term is committed by counting replicas. The leader must wait until it successfully replicates at least one entry from its current term on a majority. Once that happened, all preceding entries are logically committed as well (induction seen before).

Once a leader is established and safe, it must clean up the followers so their logs match leader's one. To do this, the leader maintains a `nextIndex` for every follower, such that it keeps track of the next index of the log (the entry) he wants to send to that follower (originally set to +1). How does the repair loop work? The leader sends an `AppendEntries` RPC with the new data, including `prevLogIndex` and `prevLogTerm`. The follower receiving this RPC check if it has this previous entry, and if not he rejects the request. If the leader receives a rejection, he decrements `nextIndex` for that follower, trying again with an older entry. Once a match is found, the follower accepts the RPC, and from there any conflicting entries following that will be removed, and the leader's truth will be appended.

Sometimes an old leader isn't dead, but it may be seen as so, leading to a new leader election. Terms are used as logical clock, and to detect stale leader. If an old leader sends a RPC, the followers (who are know in higher term), will reject it with a response, containing their term value. The old leader, seeing the higher term in the response, will step down and become a follower.

Now, how do clients interact with the cluster? The client does not know which server is the leader, so he sends the request to one of the server. If the server receiving the request is not the leader, he will redirect the request to the current leader. When the leader receives the request, he appends it to its log, send it to the replicas, and wait for a majority. Only then he will execute the command, and send an answer to the client.

Like we've seen with replication in synchronous case, a server might crash just after completing the operation but just before sending the answer. This could lead to the client trying to send again the same request (which would be executed twice, and we don't want that). So, in order to avoid this, each command has its

unique ID, and the state machine will check if a command with a specific ID has already been executed.